

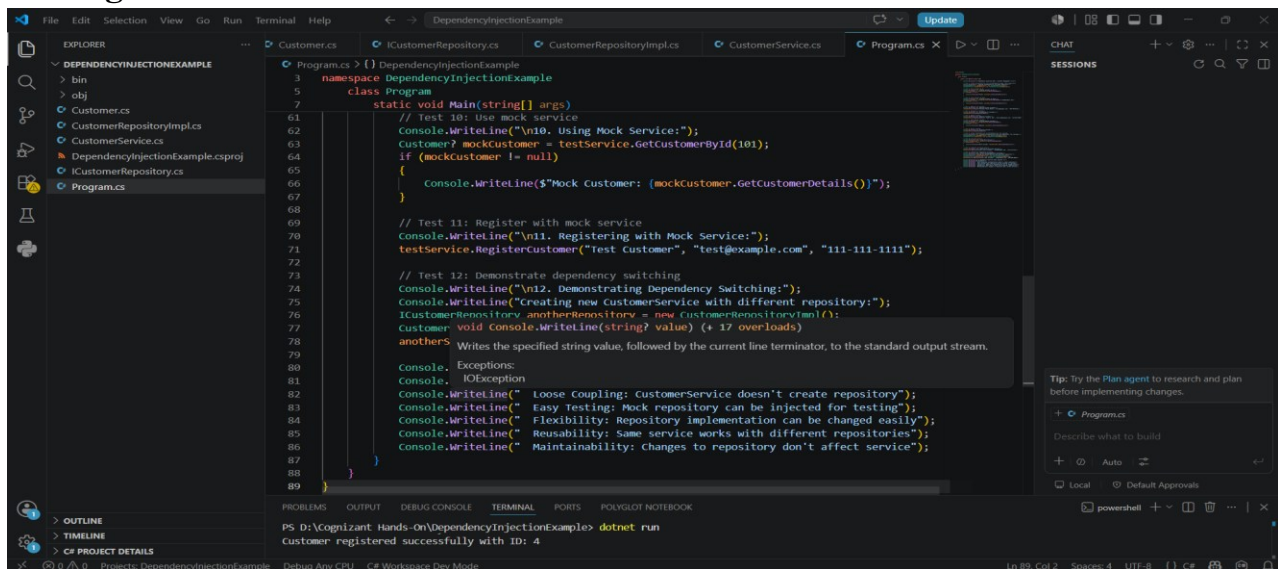
Hands-On

Exercise 11: Implementing Dependency Injection

In this module, I have been able to create, implement and test a project named `DependencyInjectionExample` with Dependency Injection in .NET. I created an interface called `CustomerRepository` with methods like `FindCustomerById()` and a concrete class `CustomerRepositoryImpl` that implements the interface. I then created a `CustomerService` class that was based on the `CustomerRepository` interface instead of any specific implementation. I have implemented the Dependency Injection using constructor-based approach to achieve loose coupling in the system by injecting the `CustomerRepository` dependency into the `CustomerService` class.

Once I deployed the application, I ran it and tested its functionality by creating an object of `CustomerService` and using `CustomerRepositoryImpl` to fetch the customer details. This hands-on activity was useful to me to grasp the benefits of Dependency Injection in terms of flexibility, maintainability and testability for .NET applications that follow the Dependency Inversion Principle which minimizes direct dependency among components.

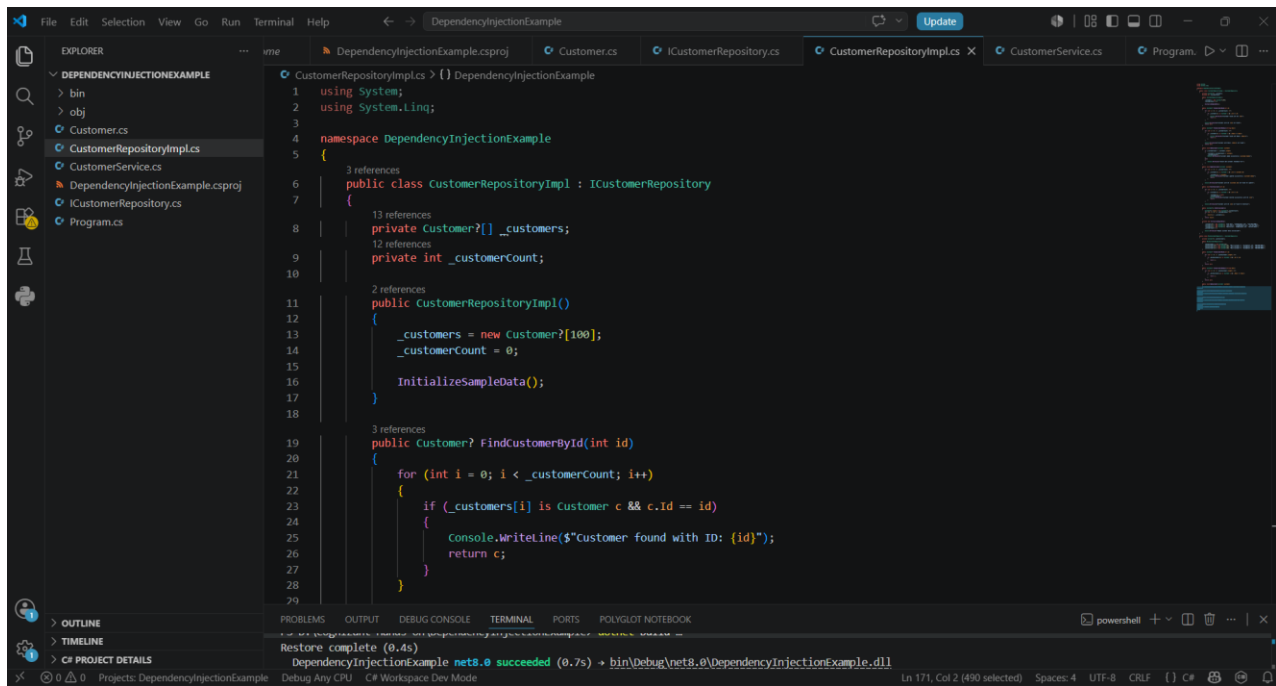
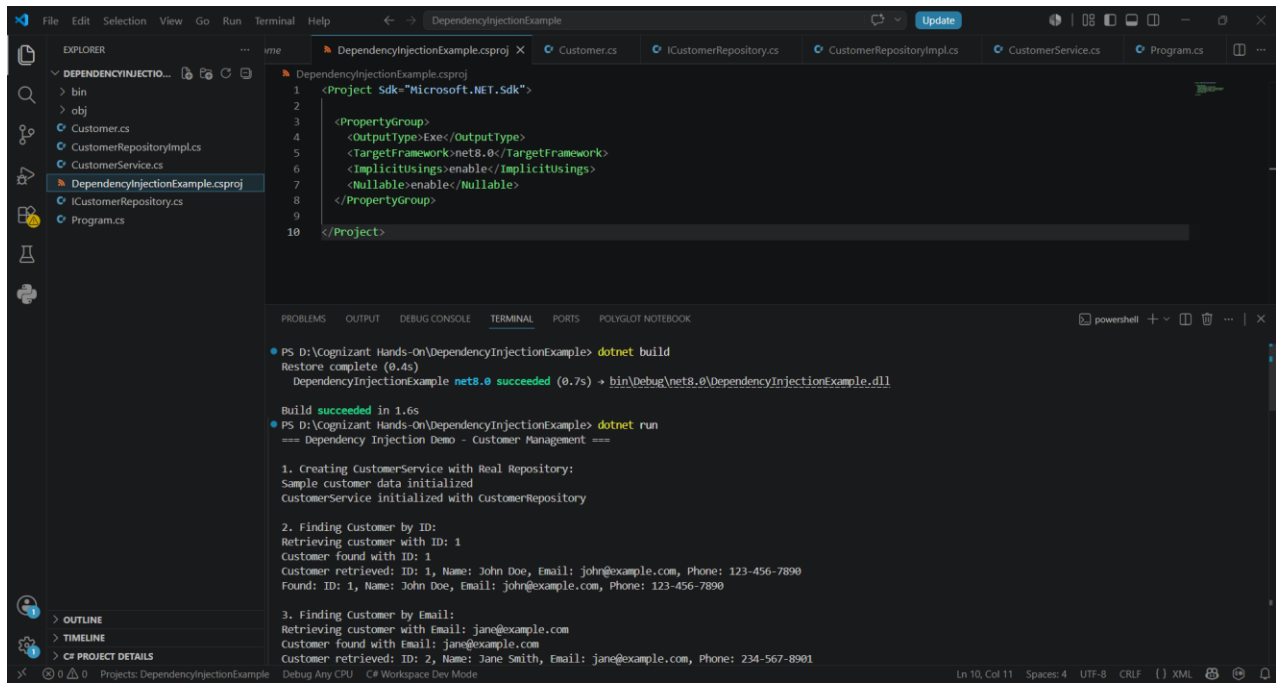
Coding:



```
1 namespace DependencyInjectionExample
2 {
3     class Program
4     {
5         static void Main(string[] args)
6         {
7             // Test 10: Use mock service
8             Console.WriteLine("\n10. Using Mock Service:");
9             Customer? mockCustomer = testService.GetCustomerById(101);
10            if (mockCustomer != null)
11            {
12                Console.WriteLine($"Mock Customer: {mockCustomer.GetCustomerDetails()}");
13            }
14
15            // Test 11: Register with mock service
16            Console.WriteLine("\n11. Registering with Mock Service:");
17            testService.RegisterCustomer("Test Customer", "test@example.com", "111-111-1111");
18
19            // Test 12: Demonstrate dependency switching
20            Console.WriteLine("\n12. Demonstrating Dependency Switching:");
21            Console.WriteLine("Creating new CustomerService with different repository:");
22            ICustomerRepository anotherRepository = new CustomerRepositoryImpl();
23            Customer? customer = testService.GetCustomerById(101);
24            Console.WriteLine($"Customer: {customer?.GetCustomerDetails()}");
25
26            Console.WriteLine("Switching to another repository (anotherRepository) and fetching customer details:");
27            customer = testService.GetCustomerById(101);
28            Console.WriteLine($"Customer: {customer?.GetCustomerDetails()}");
29
30            Console.WriteLine("Loose Coupling: CustomerService doesn't create repository");
31            Console.WriteLine("Easy testing: Mock repository can be injected for testing");
32            Console.WriteLine("Flexibility: Repository implementation can be changed easily");
33            Console.WriteLine("Reusability: Same service works with different repositories");
34            Console.WriteLine("Maintainability: Changes to repository don't affect service");
35        }
36    }
37 }
```

PS D:\Cognizant Hands-On\DependencyInjectionExample> dotnet run

Customer registered successfully with ID: 4



Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\DependencyInjectionExample> dotnet build
Restore complete (0.4s)
DependencyInjectionExample net8.0 succeeded (0.7s) -> bin\Debug\net8.0\DependencyInjectionExample.dll

Build succeeded in 1.6s
PS D:\Cognizant Hands-On\DependencyInjectionExample> dotnet run
=== Dependency Injection Demo - Customer Management ===

1. Creating CustomerService with Real Repository:
Sample customer data initialized
CustomerService initialized with CustomerRepository

2. Finding Customer by ID:
Retrieving customer with ID: 1
Customer found with ID: 1
Customer retrieved: ID: 1, Name: John Doe, Email: john@example.com, Phone: 123-456-7890
Found: ID: 1, Name: John Doe, Email: john@example.com, Phone: 123-456-7890

3. Finding Customer by Email:
Retrieving customer with Email: jane@example.com
Customer found with Email: jane@example.com
Customer retrieved: ID: 2, Name: Jane Smith, Email: jane@example.com, Phone: 234-567-8901
Found: ID: 2, Name: Jane Smith, Email: jane@example.com, Phone: 234-567-8901

4. Registering New Customer:
Registering new customer: Alice Brown
Customer added successfully: Alice Brown
Customer registered successfully with ID: 4

5. Displaying All Customers:

=== All Customers ===
ID: 1, Name: John Doe, Email: john@example.com, Phone: 123-456-7890
ID: 2, Name: Jane Smith, Email: jane@example.com, Phone: 234-567-8901
ID: 3, Name: Bob Wilson, Email: bob@example.com, Phone: 345-678-9012
ID: 4, Name: Alice Brown, Email: alice@example.com, Phone: 456-789-0123
```